TFG · SONDA PASIVA DE DETECCIÓN PQC · TAREA 4

TLScensus frente a la sonda

Instalación, ejecución sobre las mismas trazas y qué le falta respecto del TFG

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **04/11/2026** | Tutoría del miércoles 04/11/2026

1. Conclusión principal

> RESULTADO
>
> **TLScensus (v0.1.2) coincide con la sonda en 4 de los 6 casos y difiere justo en los dos que interesan al TFG:** el servidor que fuerza PQ con HelloRetryRequest (lo trata igual que uno que lo elige directamente) y el servidor que acepta clásico sin exigir PQ (culpa al cliente, cuando el servidor queda indeterminado). Clasifica **conexiones**, no equipos. Además, su tabla de grupos está fija en el código y le faltan cuatro híbridos registrados en IANA. En velocidad, ambas herramientas están en el mismo orden de magnitud.

2. Qué es y cómo se ha instalado

github.com/bear33/tlscensus · Go · Apache-2.0 · v0.1.2 (en la tutoría 1 era la v0.1.0). Existe una copia idéntica en `github.com/connectedinformation/tlscensus`: se ha comprobado que ambos repositorios tienen los mismos ficheros y el mismo último commit (`0fb97c2`, 10/10/2026).

Instalado desde el binario publicado, verificando la suma SHA-256 con `checksums.txt`:

```
curl -fsSLO
https://github.com/bear33/tlscensus/releases/latest/download/tlscensus_0.1.2_linux_amd64.tar.gz
sha256sum --ignore-missing -c checksums.txt        # tlscensus_0.1.2_linux_amd64.tar.gz: OK
tar xzf tlscensus_0.1.2_linux_amd64.tar.gz && sudo install tlscensus /usr/local/bin/
```

El nombre de fichero que indica su README (`tlscensus_Linux_x86_64.tar.gz`) no existe (error 404): el real lleva versión y arquitectura en minúsculas. Otra señal de que es un proyecto muy joven.

3. Comparación sobre `casos_clasificacion.pcap`

TLScensus asigna un único estado por conexión: `post_quantum` (se negoció PQ), `offered_not_selected` (el cliente mandó clave PQ y el servidor eligió clásico), `advertised_only` (PQ solo anunciado) o `classical`.

Caso	TLScensus	Sonda (servidor)	Valoración
1 · PQ elegido	`post_quantum`	Soporta PQ	Coinciden
2 · Cliente clásico	`classical`	Indeterminado	Coinciden; solo la sonda dice que del servidor no se sabe nada
3 · HRR fuerza PQ	**`post_quantum`**	**Fuerza PQ**	**Difieren: TLScensus no distingue este caso del 1 y solo muestra las claves del primer ClientHello (x25519)**
4 · Rechaza clave PQ	`offered_not_selected`	Inseguro	Coinciden
5 · No fuerza PQ	**`advertised_only`**	**Indeterminado**	**Difieren: TLScensus atribuye el fallo al cliente («no habrá handshake PQ»)**
6 · Servidor TLS 1.2	`offered_not_selected`	Inseguro (TLS 1.2)	Coinciden en el fondo; TLScensus no señala que la causa es la versión

Su lógica, leída en el código (`internal/inventory/inventory.go`), es: si el servidor eligió PQ, `post_quantum`; si eligió clásico, mira solo qué ofrecía el cliente. Nunca se pregunta qué hizo el servidor *con* lo que tenía, ni si pidió PQ con un HRR.

4. Velocidad sobre `rendimiento.pcap`

Herramienta	Tiempo (real)	Notas
Sonda (C + libpcap)	**0,056 s**	Lectura, reensamblado, TLS y clasificación de 3000 conexiones
TLScensus (Go, sin libpcap)	0,076 s	Hace más trabajo por conexión: JA4, ECH, avisos
Scapy (tarea 3)	19,7 s	Referencia

Con 23 000 paquetes domina el arranque y la lectura del fichero. La conclusión válida es **mismo orden de magnitud, sin desventaja para C**; una comparación de rendimiento seria requiere trazas mucho mayores.

5. Ideas a tomar y lo que le falta

Ideas útiles para la sonda	Lo que le falta respecto del TFG
• Decodificar **QUIC/HTTP3**: quien desplegó PQ pronto también desplegó HTTP/3; una sonda solo TCP subestima el uso de PQ • Detección de TLS **por contenido** en todos los puertos • Huella **JA4** (la reivindicación 10 de la patente la incluye en el inventario) • Inventario **agregado por destino** y salida estándar (CycloneDX/CBOM) • Tratamiento prudente de **ECH** (no confundir GREASE con ECH real)	• **Clasificación de equipos** (cliente y servidor por separado), que es lo que pide la patente • Distinguir el servidor que **fuerza PQ** con HRR • Estado **indeterminado** del servidor y **estado acumulado** por servidor • **Tabla de grupos actualizable** sin recompilar (reivindicación 8): la suya está en el código • Grupos IANA recientes: no reconoce **0x11E9, 0x11EA, 0x11EE, 0x11EF**; los clasificaría como clásicos • Implementación en **C sobre libpcap**, requisito del TFG

> ★ PARA COMENTAR EN LA TUTORÍA
>
> TLScensus es la herramienta más parecida y valida varias decisiones de diseño del TFG, pero **clasifica conexiones, no equipos**. Por eso falla exactamente en los dos casos clave: no distingue el servidor que **fuerza PQ con HRR** del que simplemente lo elige, y en el caso del 45 % de la tutoría 1 **culpa al cliente** cuando el servidor queda **indeterminado**. Esa diferencia, junto con el estado acumulado por servidor y la tabla de grupos actualizable, es la aportación propia del TFG frente a TLScensus.

Salida de TLScensus guardada en `medidas/tlscensus_casos_20261104.ndjson`. Otros proyectos en C revisados (pqc-flow, CPacketJA4, nDPI) se recogen en las notas de la tutoría; pqc-flow, el más cercano en C, etiqueta mal 0x11EC como «X25519Kyber768» y usa códigos 0x2001–0x2010 que no existen en IANA.